BỘ XÂY DỰNG
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP.HCM
VIỆN ĐÀO TẠO CHẤT LƯỢNG CAO

BÁO CÁO MÔN HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

Tên đề tài:

QUẢN LÝ KÝ TÚC XÁ

Giảng viên hướng dẫn:	**Trần Anh Quân**
Sinh viên thực hiện:	**MSSV:**
Nguyễn Thành Đông	083206007862
Đinh Nguyễn Thanh Duy	087204008502
Phạm Nguyễn Minh Hiếu	075206022835
Trần Cát Đằng	075208025708
Trần Khải Đăng	083206012827
Lớp:	**012012100302**

2026

BẢNG PHÂN CÔNG VÀ TỶ LỆ THAM GIA

TT	Họ tên	Tỷ lệ % tham gia	Ghi chú
1	Nguyễn Thành Đông	100%	
2	Đinh Nguyễn Thanh Duy	100%	
3	Phạm Nguyễn Minh Hiếu	100%	
4	Trần Cát Đằng	100%	
5	Trần Khải Đăng	100%	

CHƯƠNG I: TỔNG QUAN VỀ HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU & KIẾN TRÚC HỆ THỐNG QUẢN LÝ KÝ TÚC XÁ

1.1 Các khái niệm cốt lõi

- Cơ sở dữ liệu (Database – DB): là tập hợp dữ liệu có liên quan logic với nhau, được thiết kế và lưu trữ có tổ chức. CSDL của đề tài là QLKyTucXa, bao gồm thông tin tòa nhà, phòng ở, sinh viên, nhân viên, hợp đồng thuê phòng, dịch vụ tiện ích, hóa đơn, thanh toán và vi phạm nội quy của toàn bộ ký túc xá.
- Hệ quản trị CSDL (DBMS): là phần mềm trung gian — ở đây là Microsoft SQL Server — cho phép khai báo, khởi tạo, cập nhật, truy vấn và điều khiển mọi hoạt động truy xuất vào CSDL QLKyTucXa. Ứng dụng desktop Python/Tkinter của nhóm không thao tác trực tiếp lên tệp dữ liệu mà giao tiếp với DBMS qua driver ODBC (thư viện pyodbc).
- Hệ CSDL (DBS): là mô hình tổng thể gồm phần cứng máy chủ, phần mềm DBMS (SQL Server), dữ liệu (CSDL QLKyTucXa), quy trình nghiệp vụ (lập hợp đồng, lập hóa đơn, xử lý vi phạm...) và con người vận hành (ban quản lý, nhân viên, sinh viên).

1.2 Mô hình 3 mức trừu tượng dữ liệu (Three-Schema Architecture)

Để quản lý tính phức tạp của dữ liệu ký túc xá, hệ thống được phân chia thành 3 mức trừu tượng:

1. Mức Vật lý (Physical Level): SQL Server tổ chức lưu trữ 12 bảng của CSDL QLKyTucXa trên đĩa. Khóa chính như MaSV, MaPhong, MaHopDong, MaHoaDon được đánh chỉ mục (clustered index) để tăng tốc tìm kiếm và JOIN giữa các bảng liên quan (vd Phong – HopDong – HoaDon).
2. Mức Luận lý (Logical Level): mô tả toàn bộ 12 bảng, các ràng buộc toàn vẹn (Primary Key, Foreign Key, Check Constraint) được khai báo tường minh trong sql/SQL.sql, ví dụ Phong.GioiTinhPhong CHECK IN (N'Nam', N'Nữ'), HopDong CHECK (NgayKetThucDK > NgayBatDau).
3. Mức Khung nhìn (View Level): cung cấp giao diện dữ liệu tùy theo vai trò — sinh viên (qua cổng tra cứu) chỉ nên thấy hợp đồng, hóa đơn của chính mình và danh sách phòng còn trống (view vw_DanhSachPhongTrong); trong khi nhân viên/ban quản lý thấy toàn bộ dữ liệu qua màn hình Dashboard (hàm get_dashboard_data trong db/repository.py) — thống kê số phòng, tỷ lệ lấp đầy, doanh thu, công nợ và vi phạm.

1.3 Tính độc lập dữ liệu (Data Independence)

- **Tính độc lập luận lý (Logical Data Independence):** khả năng thay đổi sơ đồ luận lý mà không cần sửa các ứng dụng phía trên.
    - Ví dụ trong QLKyTucXa: khi bổ sung thêm cột SoDienThoaiPhuHuynh vào bảng SinhVien, các hàm truy vấn hiện có như get_all_sinh_vien() hay thủ tục sp_ThemHopDong hoàn toàn không bị ảnh hưởng, không cần viết lại.

- **Tính độc lập vật lý (Physical Data Independence):** khả năng thay đổi sơ đồ vật lý mà không làm thay đổi sơ đồ luận lý hay câu truy vấn ứng dụng.
    - Ví dụ trong QLKyTucXa: khi quản trị viên tạo thêm Non-Clustered Index trên cột Phong.TinhTrang để tăng tốc lọc phòng trống, hoặc chuyển tệp CSDL sang ổ SSD mới, cấu trúc bảng và toàn bộ câu lệnh T-SQL trong ứng dụng vẫn giữ nguyên.

CHƯƠNG II: LẬP TRÌNH T-SQL NÂNG CAO TRONG HỆ THỐNG QLKyTucXa

2.1 Cấu trúc dữ liệu hệ thống

CSDL QLKyTucXa (chi tiết đầy đủ trong sql/SQL.sql) gồm 12 bảng nghiệp vụ, 4 view báo cáo, 4 stored procedure và 2 trigger:

- ToaNha, LoaiPhong, Phong — quản lý cơ sở vật chất.
- SinhVien, NhanVien — hồ sơ con người.
- HopDong — hợp đồng thuê phòng (liên kết SinhVien, Phong, NhanVien).
- DichVu, ChiSoDichVu — dịch vụ tiện ích (điện, nước, wifi...) và chỉ số sử dụng theo tháng.
- HoaDon, ChiTietHoaDon, ThanhToan — tài chính: lập hóa đơn, chi tiết và các lần thanh toán.
- ViPham — vi phạm nội quy của sinh viên.

2.2 Xử lý điều khiển luồng dữ liệu nâng cao

a) Xử lý logic nghiệp vụ bằng IF...ELSE kết hợp TRANSACTION

Kịch bản: khi sinh viên trả phòng (mở rộng thủ tục sp_TraPhong), nếu ngày trả thực tế trễ hơn ngày kết thúc dự kiến trong hợp đồng thì tự động tính phí phạt trả trễ 50.000đ/ngày.

```
DECLARE @MaHopDong CHAR(10) = 'HD00000010';
DECLARE @NgayTraThucTe DATE = CAST(GETDATE() AS DATE);
DECLARE @NgayKetThucDK DATE;
DECLARE @SoNgayTre INT;
DECLARE @PhiPhat DECIMAL(12,0) = 0;

SELECT @NgayKetThucDK = NgayKetThucDK FROM HopDong WHERE MaHopDong = @MaHopDong;
SET @SoNgayTre = DATEDIFF(DAY, @NgayKetThucDK, @NgayTraThucTe);

BEGIN TRAN;
   IF @SoNgayTre > 0
   BEGIN
      SET @PhiPhat = @SoNgayTre * 50000;
      PRINT N'Trả phòng trễ ' + CAST(@SoNgayTre AS VARCHAR) + N' ngày. Phí phạt: '
          + CAST(@PhiPhat AS VARCHAR) + N' VNĐ.';
   END
   ELSE
      PRINT N'Trả phòng đúng hạn hoặc sớm hơn. Không phát sinh phí phạt.';

   UPDATE HopDong
   SET NgayTraThucTe = @NgayTraThucTe, TrangThai = N'Đã kết thúc'
   WHERE MaHopDong = @MaHopDong;
COMMIT TRAN;
```

b) Phân loại sinh viên tự động bằng CASE...WHEN

Dựa vào số lần vi phạm nội quy (bảng ViPham), hệ thống tự động gắn nhãn cảnh báo cho từng sinh viên:

```
SELECT
    sv.MaSV,
    sv.HoTen,
    SoLanViPham = COUNT(vp.MaVP),
    XepLoaiYThuc = CASE
        WHEN COUNT(vp.MaVP) = 0 THEN N'Chưa vi phạm'
        WHEN COUNT(vp.MaVP) BETWEEN 1 AND 2 THEN N'Nhắc nhở'
        WHEN COUNT(vp.MaVP) >= 3 THEN N'Cảnh cáo — xem xét chấm dứt hợp đồng'
    END
FROM SinhVien sv
LEFT JOIN ViPham vp ON vp.MaSV = sv.MaSV
GROUP BY sv.MaSV, sv.HoTen;
```

c) Vòng lặp WHILE rà soát tình trạng phòng theo từng tòa

```
DECLARE @Counter INT = 1;
DECLARE @TongPhong INT;
DECLARE @MaPhongHienTai CHAR(6);

SELECT @TongPhong = COUNT(*) FROM Phong WHERE MaToa = 'TN001';

WHILE @Counter <= @TongPhong
BEGIN
    SELECT @MaPhongHienTai = MaPhong FROM (
        SELECT MaPhong, ROW_NUMBER() OVER (ORDER BY MaPhong) AS rn
        FROM Phong WHERE MaToa = 'TN001'
    ) t WHERE rn = @Counter;

    PRINT N'Đang kiểm tra tình trạng phòng: ' + @MaPhongHienTai;
    SET @Counter = @Counter + 1;
END;
```

2.3 Lập trình Cursor xử lý nhắc nợ hóa đơn quá hạn

Khi cần duyệt qua từng hóa đơn quá hạn để soạn nội dung nhắc nợ riêng cho từng sinh viên — việc mà câu lệnh tập hợp (set-based) khó thực hiện linh hoạt — ta dùng Cursor:

```
DECLARE @Cur_MaHoaDon CHAR(10), @Cur_MaSV CHAR(10), @Cur_HoTen NVARCHAR(100);
DECLARE @Cur_TongTien DECIMAL(12,0), @Cur_HanThanhToan DATE;
DECLARE @SoNgayQuaHan INT;

DECLARE cur_HoaDonQuaHan CURSOR FOR
    SELECT hd.MaHoaDon, sv.MaSV, sv.HoTen, hd.TongTien, hd.HanThanhToan
    FROM HoaDon hd
    JOIN HopDong h ON hd.MaHopDong = h.MaHopDong
    JOIN SinhVien sv ON h.MaSV = sv.MaSV
    WHERE hd.TrangThai <> N'Đã thanh toán' AND hd.HanThanhToan < GETDATE();

OPEN cur_HoaDonQuaHan;
```

```
FETCH NEXT FROM cur_HoaDonQuaHan
   INTO @Cur_MaHoaDon, @Cur_MaSV, @Cur_HoTen, @Cur_TongTien, @Cur_HanThanhToan;

WHILE @@FETCH_STATUS = 0
BEGIN
   SET @SoNgayQuaHan = DATEDIFF(DAY, @Cur_HanThanhToan, GETDATE());
   PRINT N'=== NHẮC NỢ ===';
   PRINT N'Sinh viên: ' + @Cur_HoTen + N' (' + @Cur_MaSV + N')';
   PRINT N'Hóa đơn: ' + @Cur_MaHoaDon + N' — Số tiền: ' + CAST(@Cur_TongTien AS VARCHAR);
   PRINT N'Quá hạn: ' + CAST(@SoNgayQuaHan AS VARCHAR) + N' ngày.';
   PRINT N'-------------------------';

   FETCH NEXT FROM cur_HoaDonQuaHan
      INTO @Cur_MaHoaDon, @Cur_MaSV, @Cur_HoTen, @Cur_TongTien, @Cur_HanThanhToan;
END;

CLOSE cur_HoaDonQuaHan;
DEALLOCATE cur_HoaDonQuaHan;
```

CHƯƠNG III: MÔ HÌNH HÓA VÀ QUẢN LÝ GIAO TÁC (TRANSACTION MANAGEMENT)

3.1 Phân tích 4 tính chất ACID của giao tác trong hệ thống QLKyTucXa

Một giao tác T trong hệ thống KTX đại diện cho một đơn vị xử lý logic (Logical Unit of Work), ví dụ giao tác "lập hợp đồng thuê phòng mới" (thủ tục sp_ThemHopDong).

1. Tính Nguyên tố (Atomicity): giao tác lập hợp đồng gồm [sinh mã hợp đồng mới] + [INSERT vào HopDong] + [trigger kiểm tra & cập nhật TinhTrang của Phong]. Nếu bất kỳ bước nào thất bại (ví dụ trigger phát hiện phòng đã đủ sức chứa), toàn bộ giao tác phải bị hủy bỏ hoàn toàn (ROLLBACK), không được phép chỉ tạo hợp đồng mà bỏ qua việc cập nhật trạng thái phòng.
2. Tính Nhất quán (Consistency): giao tác đưa CSDL từ trạng thái hợp lệ này sang trạng thái hợp lệ khác. Trong KTX, ràng buộc "số hợp đồng đang hiệu lực trong 1 phòng không được vượt SucChua" và "giới tính sinh viên phải khớp GioiTinhPhong" (do trigger trg_KiemTraHopDong đảm nhiệm) không bao giờ được phép vi phạm, dù có bao nhiêu giao tác chạy song song.
3. Tính Cô lập (Isolation): các giao tác thực thi đồng thời (ví dụ nhiều nhân viên trực quầy cùng lập hợp đồng, cùng thu tiền vào giờ cao điểm đầu năm học) không được can thiệp lẫn nhau; kết quả của giao tác T chỉ được các giao tác khác nhìn thấy sau khi T đã COMMIT. Đây chính là tính chất không được đảm bảo trọn vẹn nếu hệ thống thiếu cơ chế khóa phù hợp — nội dung phân tích chi tiết ở Chương IV.
4. Tính Bền vững (Durability): khi một hợp đồng hoặc hóa đơn đã báo lập thành công (COMMIT), dữ liệu phải được lưu vĩnh viễn, kể cả khi hệ thống gặp sự cố ngay sau đó.

3.2 Kịch bản Stored Procedure giao tác toàn diện: sp_ThemHopDong

Thủ tục lập hợp đồng thuê phòng dưới đây (trích từ sql/SQL.sql, có bổ sung khóa UPDLOCK để chuẩn bị cho phần chống Lost Update ở Chương IV) minh họa đầy đủ mô hình TRY/CATCH kết hợp TRANSACTION:

```
CREATE PROCEDURE sp_ThemHopDong
   @MaSV CHAR(10), @MaPhong CHAR(6), @MaNV CHAR(6),
   @NgayBatDau DATE, @NgayKetThucDK DATE
AS
BEGIN
   SET NOCOUNT ON;
   BEGIN TRY
      BEGIN TRAN;
         -- Khóa cập nhật (UPDLOCK) trên dòng Phong để tránh 2 giao tác
         -- cùng đọc thấy phòng còn chỗ rồi cùng chèn hợp đồng (xem Chương IV)
         IF NOT EXISTS (
            SELECT 1 FROM Phong WITH (UPDLOCK, ROWLOCK)
            WHERE MaPhong = @MaPhong AND TinhTrang <> N'Bảo trì'
         )
         BEGIN
```

```
            RAISERROR(N'Phòng không tồn tại hoặc đang bảo trì.', 16, 1);
            ROLLBACK TRAN; RETURN;
        END

        DECLARE @MaHopDongMoi CHAR(10);
        SELECT @MaHopDongMoi = 'HD' + RIGHT('000000' +
            CAST(ISNULL(MAX(CAST(SUBSTRING(MaHopDong,3,8) AS INT)),0) + 1 AS VARCHAR), 8)
        FROM HopDong;

        INSERT INTO HopDong (MaHopDong, MaSV, MaPhong, MaNV, NgayBatDau, NgayKetThucDK)
        VALUES (@MaHopDongMoi, @MaSV, @MaPhong, @MaNV, @NgayBatDau, @NgayKetThucDK);
        -- trigger trg_KiemTraHopDong + trg_CapNhatTinhTrangPhong tự chạy tại đây
      COMMIT TRAN;
      PRINT N'Lập hợp đồng thành công: ' + @MaHopDongMoi;
    END TRY
    BEGIN CATCH
      IF @@TRANCOUNT > 0 ROLLBACK TRAN;
      THROW;
    END CATCH
END;
```

CHƯƠNG IV: NĂM VẤN ĐỀ THƯỜNG GẶP KHI TRUY CẬP ĐỒNG THỜI TRONG HỆ THỐNG QLKyTucXa

4.1 Đặt vấn đề

Vào các thời điểm cao điểm như đầu năm học (nhiều sinh viên cùng làm thủ tục nhận phòng) hoặc cuối tháng (nhiều nhân viên cùng ghi chỉ số dịch vụ, lập hóa đơn, thu tiền), CSDL QLKyTucXa bị nhiều giao tác truy cập đồng thời. Nếu không kiểm soát tương tranh (concurrency control) chặt chẽ, 5 vấn đề sau có thể xảy ra và làm sai lệch dữ liệu nghiệp vụ:

Vấn đề tương tranh	Tình huống trong QLKyTucXa	Hậu quả
Mất dữ liệu cập nhật (Lost Update)	2 nhân viên cùng ghi chỉ số điện/nước (ChiSoDichVu) cho cùng 1 phòng, cùng tháng.	Một trong hai lần ghi bị ghi đè hoàn toàn, không có cảnh báo — hóa đơn tính sai tiền dịch vụ.
Đọc dữ liệu rác (Dirty Read)	Nhân viên khác đọc hóa đơn đang được sp_LapHoaDon tạo dở (chưa COMMIT) rồi bị ROLLBACK.	Thông báo công nợ sai cho sinh viên dựa trên dữ liệu chưa từng tồn tại chính thức.
Không đọc lại được dữ liệu (Non-repeatable Read)	Kế toán đọc trạng thái 1 hóa đơn 2 lần trong cùng 1 giao dịch đối chiếu; giữa 2 lần đọc có người khác thanh toán hóa đơn đó.	Hai lần đọc cùng một dòng dữ liệu trong cùng giao dịch cho 2 kết quả khác nhau.
Bóng ma (Phantom Read)	Ban quản lý đếm số phòng trống trong 1 tòa 2 lần trong cùng giao dịch kiểm phòng; giữa 2 lần đọc có hợp đồng mới được lập làm đổi trạng thái 1 phòng.	Tập kết quả của cùng 1 truy vấn thay đổi giữa 2 lần chạy trong cùng giao dịch.
Bế tắc (Deadlock)	Giao tác "chuyển phòng" và giao tác "lập hóa đơn" cùng khóa chéo bảng Phong và HopDong theo 2 thứ tự ngược nhau.	Cả hai giao tác chờ nhau vô thời hạn; SQL Server phải tự hủy một giao tác để phá vỡ bế tắc.

4.2 Phân tích chi tiết và minh chứng từng vấn đề

4.2.1 Mất dữ liệu cập nhật (Lost Update)

Định nghĩa (theo giáo trình môn học): xảy ra khi hai giao tác cùng đọc một đơn vị dữ liệu, sau đó cùng cập nhật dựa trên giá trị đã đọc; giao tác COMMIT sau sẽ ghi đè hoàn toàn kết quả của giao tác COMMIT trước, làm mất cập nhật đó mà hệ thống không hề báo lỗi.

Cuối tháng 9/2026, nhân viên NV1 và NV2 cùng được giao ghi chỉ số điện tháng cho phòng TN001-01 (bản ghi ChiSoDichVu có MaCS = 500, ChiSoDau = 120, ChiSoCuoi đang NULL). NV1 xuống ghi số trực tiếp đọc được 245, NV2 (do check chéo từ sổ tay) đọc được 254. Cả hai cùng mở ứng dụng gần như đồng thời và cùng cập nhật.

Minh chứng bằng kịch bản T-SQL (mô phỏng 2 phiên làm việc song song trong SQL Server Management Studio — mở 2 cửa sổ Query, dùng WAITFOR DELAY để ép hai giao tác xen kẽ đúng như thực tế):

```
-- SESSION A (NV1)
BEGIN TRAN;
   SELECT ChiSoCuoi FROM ChiSoDichVu WHERE MaCS = 500;   -- đọc: NULL
   WAITFOR DELAY '00:00:05';
   UPDATE ChiSoDichVu SET ChiSoCuoi = 245 WHERE MaCS = 500;
COMMIT TRAN;

-- SESSION B (NV2) — chạy gần như đồng thời với Session A
BEGIN TRAN;
   SELECT ChiSoCuoi FROM ChiSoDichVu WHERE MaCS = 500;   -- đọc: NULL (chưa thấy A ghi)
   UPDATE ChiSoDichVu SET ChiSoCuoi = 254 WHERE MaCS = 500;
COMMIT TRAN;

-- Kết quả cuối: ChiSoCuoi = 254 (giá trị của B, COMMIT sau).
-- Giá trị 245 mà A vừa ghi bị B ghi đè hoàn toàn — không hề có cảnh báo lỗi.
```

Hậu quả: Hóa đơn tiền điện tháng 9 của phòng TN001-01 sẽ được tính theo chỉ số sai (254 thay vì giá trị đúng), gây thất thoát hoặc tính nhầm tiền cho sinh viên mà không ai phát hiện.

Giải pháp khắc phục: Dùng khóa cập nhật tường minh khi đọc trước khi ghi: SELECT ... WITH (UPDLOCK, ROWLOCK) WHERE MaCS = 500; — khi đó Session B phải chờ Session A COMMIT xong mới được đọc/ghi, tránh tình trạng "đọc mù" trên dữ liệu sắp bị người khác thay đổi.

4.2.2 Đọc dữ liệu rác (Dirty Read)

Định nghĩa (theo giáo trình môn học): xảy ra khi một giao tác đọc phải dữ liệu do một giao tác khác vừa ghi nhưng CHƯA COMMIT; nếu giao tác ghi sau đó bị ROLLBACK, dữ liệu vừa đọc trở thành "rác" vì chưa từng tồn tại chính thức.

Nhân viên NV1 đang chạy sp_LapHoaDon để lập hóa đơn tháng 9 cho hợp đồng HD00000001: đã INSERT xong dòng HoaDon (TienDichVu tạm ghi 0) nhưng giao tác CHƯA COMMIT (còn đang chèn tiếp ChiTietHoaDon). Ngay lúc đó NV2 cố tình dùng gợi ý NOLOCK để xem nhanh công nợ và báo cho sinh viên.

Minh chứng bằng kịch bản T-SQL (mô phỏng 2 phiên làm việc song song trong SQL Server Management Studio — mở 2 cửa sổ Query, dùng WAITFOR DELAY để ép hai giao tác xen kẽ đúng như thực tế):

```
-- SESSION A (NV1) — đang lập hóa đơn, transaction còn mở
BEGIN TRAN;
   INSERT INTO HoaDon (MaHoaDon, MaHopDong, Thang, Nam, TienPhong, TienDichVu, HanThanhToan)
   VALUES ('HO00000099','HD00000001',9,2026,1200000,0,'2026-10-10');
   WAITFOR DELAY '00:00:05';   -- đang xử lý tiếp ChiTietHoaDon...
   -- giả sử phát sinh lỗi ở bước tiếp theo
```

```
ROLLBACK TRAN;   -- hủy toàn bộ hóa đơn vừa tạo

-- SESSION B (NV2) — cố tình đọc kiểu "bẩn" trong lúc A chưa COMMIT
SELECT * FROM HoaDon WITH (NOLOCK)
WHERE MaHopDong = 'HD00000001' AND Thang = 9;
-- B đọc được dòng HO00000099 dù A CHƯA COMMIT, báo sinh viên "chỉ nợ 1.200.000đ tiền phòng"
-- vài giây sau A ROLLBACK -> hóa đơn đó CHƯA TỪNG tồn tại chính thức -> B vừa báo tin rác
```

Hậu quả: Sinh viên nhận thông báo công nợ sai (số tiền chưa đầy đủ hoặc dòng hóa đơn không hề tồn tại), gây nhầm lẫn và mất uy tín trong khâu thu học phí ký túc xá.

Giải pháp khắc phục: SQL Server mặc định dùng mức cô lập READ COMMITTED, tự động ngăn dirty read (chỉ cho đọc dữ liệu đã COMMIT). Vấn đề chỉ phát sinh khi nhân viên/lập trình viên chủ động dùng gợi ý NOLOCK hoặc SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED — vì vậy quy ước của nhóm là KHÔNG dùng NOLOCK cho bất kỳ thao tác liên quan tài chính (HoaDon, ThanhToan).

4.2.3 Không đọc lại được dữ liệu (Non-repeatable Read)

Định nghĩa (theo giáo trình môn học): xảy ra khi một giao tác đọc cùng một dữ liệu hai lần; giữa hai lần đọc, một giao tác khác đã cập nhật và COMMIT dữ liệu đó, khiến hai lần đọc trong cùng một giao tác cho kết quả khác nhau.

Kế toán đang thực hiện một giao dịch đối chiếu công nợ dài: đọc trạng thái hóa đơn HO00000005 ở đầu giao dịch, xử lý một số thao tác khác, rồi đọc lại để in phiếu xác nhận cuối giao dịch. Giữa hai lần đọc đó, bộ phận thu ngân ghi nhận thanh toán cho đúng hóa đơn này.

Minh chứng bằng kịch bản T-SQL (mô phỏng 2 phiên làm việc song song trong SQL Server Management Studio — mở 2 cửa sổ Query, dùng WAITFOR DELAY để ép hai giao tác xen kẽ đúng như thực tế):

```
-- SESSION A (Kế toán) — mức cô lập mặc định READ COMMITTED
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRAN;
   SELECT TrangThai, TongTien FROM HoaDon WHERE MaHoaDon = 'HO00000005';
   -- lần đọc 1: TrangThai = N'Chưa thanh toán'
   WAITFOR DELAY '00:00:05';
   SELECT TrangThai, TongTien FROM HoaDon WHERE MaHoaDon = 'HO00000005';
   -- lần đọc 2: TrangThai = N'Đã thanh toán'  <-- khác lần đọc 1!
COMMIT TRAN;

-- SESSION B (Thu ngân) — chạy xen giữa 2 lần đọc của A, COMMIT ngay
EXEC sp_ThanhToanHoaDon @MaHoaDon = 'HO00000005', @SoTien = 1200000;
```

Hậu quả: Trong cùng một giao dịch đối chiếu, kế toán nhận hai kết quả khác nhau cho cùng một câu hỏi — phiếu xác nhận in ra có thể không khớp với báo cáo đã lập trước đó trong cùng phiên làm việc.

Giải pháp khắc phục: Nâng mức cô lập lên REPEATABLE READ: SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; — SQL Server giữ khóa chia sẻ (Shared Lock) trên các dòng đã đọc cho đến khi Session A COMMIT, buộc Session B phải chờ.

4.2.4 Bóng ma (Phantom Read)

Định nghĩa (theo giáo trình môn học): xảy ra khi một giao tác đang đọc một tập hợp dữ liệu theo điều kiện; một giao tác khác thêm hoặc xóa (hoặc làm đổi trạng thái) một đơn vị dữ liệu thỏa điều kiện đó, khiến tập kết quả của cùng một truy vấn thay đổi giữa các lần đọc trong cùng giao tác.

Ban quản lý đang kiểm tra số phòng trống của tòa TN001 để phân bổ chỗ ở đầu năm học — chạy cùng một câu truy vấn hai lần trong một giao dịch REPEATABLE READ. Giữa hai lần đọc, nhân viên lễ tân lập hợp đồng mới cho một sinh viên vào đúng 1 trong các phòng đang "trống" đó; trigger trg_CapNhatTinhTrangPhong lập tức đổi trạng thái phòng này.

Minh chứng bằng kịch bản T-SQL (mô phỏng 2 phiên làm việc song song trong SQL Server Management Studio — mở 2 cửa sổ Query, dùng WAITFOR DELAY để ép hai giao tác xen kẽ đúng như thực tế):

```
-- SESSION A (Ban quản lý) — REPEATABLE READ vẫn CHƯA đủ để chống phantom
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRAN;
   SELECT MaPhong FROM Phong WHERE MaToa = 'TN001' AND TinhTrang = N'Trống';
   -- lần đọc 1: trả về 5 phòng, trong đó có TN001-05
   WAITFOR DELAY '00:00:05';
   SELECT MaPhong FROM Phong WHERE MaToa = 'TN001' AND TinhTrang = N'Trống';
   -- lần đọc 2: chỉ còn 4 phòng — TN001-05 đã "biến mất" khỏi tập kết quả
COMMIT TRAN;

-- SESSION B (Lễ tân) — lập hợp đồng cho SV mới vào phòng TN001-05, COMMIT ngay
EXEC sp_ThemHopDong
   @MaSV = 'SV00000123', @MaPhong = 'TN001-05', @MaNV = 'NV0001',
   @NgayBatDau = '2026-09-14', @NgayKetThucDK = '2027-06-30';
-- trigger trg_CapNhatTinhTrangPhong tự đổi TinhTrang của TN001-05 -> N'Đang sử dụng'
```

Hậu quả: REPEATABLE READ chỉ khóa các DÒNG đã đọc ở lần trước, không khóa "khoảng điều kiện" WHERE TinhTrang = N'Trống', nên một dòng thỏa/không còn thỏa điều kiện vẫn có thể xuất hiện hoặc biến mất giữa hai lần đọc — Ban quản lý dựa vào lần đọc đầu (5 phòng) có thể phân bổ nhầm cho sinh viên vào phòng TN001-05 dù phòng này vừa được người khác nhận.

Giải pháp khắc phục: Nâng mức cô lập lên SERIALIZABLE: SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; — SQL Server khóa toàn bộ khoảng giá trị thỏa điều kiện WHERE (range lock), chặn cả thao tác làm đổi tập kết quả (INSERT/UPDATE liên quan) cho đến khi Session A COMMIT.

4.2.5 Bế tắc (Deadlock)

Định nghĩa (theo giáo trình môn học): một hệ thống rơi vào trạng thái deadlock nếu tồn tại một tập giao tác mà mỗi giao tác trong đó đang chờ một hạng mục dữ liệu bị giữ bởi một giao tác khác cũng trong tập — tạo thành một chu trình chờ khép kín khiến không giao tác nào có thể tiếp tục.

Nhân viên NV1 thực hiện nghiệp vụ "chuyển phòng" cho sinh viên: cập nhật hợp đồng cũ (khóa bảng HopDong trước) rồi mới cập nhật trạng thái phòng (khóa bảng Phong sau). Cùng lúc đó, nhân viên NV2 thực hiện nghiệp vụ "lập hóa đơn" theo thứ tự khóa ngược lại: cập nhật phòng trước (để giữ chỗ tính giá) rồi mới cập nhật hợp đồng liên quan.

Minh chứng bằng kịch bản T-SQL (mô phỏng 2 phiên làm việc song song trong SQL Server Management Studio — mở 2 cửa sổ Query, dùng WAITFOR DELAY để ép hai giao tác xen kẽ đúng như thực tế):

```
-- SESSION A (NV1 — chuyển phòng: HopDong trước, Phong sau)
BEGIN TRAN;
   UPDATE HopDong SET TrangThai = N'Đã kết thúc'
   WHERE MaHopDong = 'HD00000010';          -- (1) khóa X trên HopDong
   WAITFOR DELAY '00:00:05';
   UPDATE Phong SET TinhTrang = N'Trống'
   WHERE MaPhong = 'TN001-05';              -- (2) CẦN khóa X trên Phong
COMMIT TRAN;

-- SESSION B (NV2 — lập hóa đơn: Phong trước, HopDong sau) — chạy gần như đồng thời
BEGIN TRAN;
   UPDATE Phong SET TinhTrang = N'Đang sử dụng'
   WHERE MaPhong = 'TN001-05';              -- (1) khóa X trên Phong
   WAITFOR DELAY '00:00:05';
   UPDATE HopDong SET MaNV = 'NV0003'
   WHERE MaHopDong = 'HD00000010';          -- (2) CẦN khóa X trên HopDong
COMMIT TRAN;

-- A đang giữ khóa HopDong, chờ khóa Phong (mà B đang giữ)
-- B đang giữ khóa Phong, chờ khóa HopDong (mà A đang giữ)  =>  DEADLOCK
-- SQL Server (Lock Monitor) phát hiện chu trình chờ và trả lỗi:
-- Msg 1205, Level 13: Transaction (Process ID ..) was deadlocked on lock
-- resources with another process and has been chosen as the deadlock victim.
-- Rerun the transaction.
```

Hậu quả: Một trong hai giao tác (chọn ngẫu nhiên theo chi phí rollback thấp nhất) bị SQL Server tự động hủy và ROLLBACK toàn bộ; nghiệp vụ chuyển phòng hoặc lập hóa đơn của nhân viên đó thất bại giữa chừng và phải làm lại.

Giải pháp khắc phục: (1) Quy ước truy cập bảng theo CÙNG một thứ tự cố định trong mọi thủ tục — ví dụ luôn khóa Phong trước rồi mới đến HopDong; (2) rút ngắn thời gian giữ khóa (giao tác gọn, không WAITFOR/chờ người dùng giữa các câu lệnh); (3) bắt lỗi 1205 trong khối BEGIN CATCH và tự động thử lại giao tác (retry logic) ở tầng ứng dụng.

4.3 Phân tích Khả tuần tự Xung đột bằng Precedence Graph

Giả sử hệ thống nhận một lịch giao tác S thực thi các thao tác đọc/ghi trên phòng P (MaPhong) và hợp đồng H (HopDong) bởi hai giao tác T1 (chuyển phòng) và T2 (lập hóa đơn):

```
S: r1(P); w1(P); r2(P); w2(P); r1(H); w1(H); r2(H); w2(H)
```

Thuật toán xây dựng đồ thị tuần tự P(S):

1. Tạo các đỉnh (nodes) đại diện cho T1, T2.
2. Xét các cặp thao tác xung đột — hai thao tác xung đột nếu thuộc hai giao tác khác nhau, truy xuất cùng một mục dữ liệu và có ít nhất một thao tác Write:
    - w1(P) trước r2(P) trên P ⇒ T1 → T2
    - w1(P) trước w2(P) trên P ⇒ T1 → T2
    - w1(H) trước r2(H) trên H ⇒ T1 → T2
    - w1(H) trước w2(H) trên H ⇒ T1 → T2

Kết luận: đồ thị P(S) chỉ có một chiều cung T1 → T2, không chứa chu trình (no cycles). Do đó lịch S là Khả tuần tự xung đột (Conflict Serializable) và tương đương với lịch tuần tự <T1, T2> — đây chính là kết quả mà giải pháp "luôn khóa theo cùng thứ tự Phong → HopDong" ở mục 4.2.5 hướng tới: khi cả hai giao tác cùng tuân theo một thứ tự truy cập nhất quán, đồ thị tuần tự sẽ không còn chu trình và deadlock không thể xảy ra.

4.4 Phân tích Khả tuần tự View bằng PolyGraph

Xét lịch S' không khả tuần tự xung đột trên cùng mục dữ liệu Phong.TinhTrang: S': r1(P); w2(P); w1(P); w3(P)

1. Bổ sung giao tác đầu Tb (ghi giá trị khởi tạo N'Trống') và giao tác cuối Tf (đọc giá trị cuối cùng).
2. Xác định các quan hệ Read-From: T1 đọc giá trị của Tb.
3. Tạo các cung ép buộc đảm bảo thứ tự đọc/ghi giá trị không bị phá vỡ, rồi loại bỏ các cung vi phạm.
4. Nếu tồn tại một cách chọn hướng cho tất cả các cung đa hướng sao cho đồ thị thu được không chứa chu trình thì lịch đó khả tuần tự View.

Kết quả: lịch S' đạt tiêu chuẩn View Serializable — dù không khả tuần tự xung đột, kết quả cuối cùng của Phong.TinhTrang vẫn tương đương một lịch tuần tự nào đó.

CHƯƠNG V: CÁC GIAO THỨC ĐIỀU KHIỂN TƯƠNG TRANH

5.1 Giao thức Khóa hai giai đoạn (Two-Phase Locking – 2PL)

Quy tắc vận hành 2PL:

- Giai đoạn Mở rộng (Growing Phase): giao tác có thể xin cấp khóa Shared (S) hoặc Exclusive (X), nhưng không được giải phóng bất kỳ khóa nào.
- Giai đoạn Thu hẹp (Shrinking Phase): giao tác bắt đầu giải phóng khóa, nhưng từ thời điểm này không được xin cấp thêm bất kỳ khóa nào khác.

Biến thể Strict 2PL & Rigorous 2PL áp dụng cho QLKyTucXa:

- Strict 2PL: giữ toàn bộ khóa Exclusive (X) cho đến khi giao tác COMMIT hoặc ROLLBACK hẳn — đây chính là cơ chế mặc định của SQL Server, tránh hiện tượng Cascading Rollback (hủy dây chuyền) khi 1 hợp đồng bị rollback kéo theo các thao tác đã đọc dữ liệu đó.
- Rigorous 2PL: giữ TOÀN BỘ khóa (cả S và X) cho đến khi kết thúc giao tác — đúng như cách SET TRANSACTION ISOLATION LEVEL SERIALIZABLE hoạt động trong ví dụ chống Phantom Read ở mục 4.2.4.

Kịch bản minh họa Strict 2PL khi lập hợp đồng:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
   -- Xin cấp khóa Read Lock trên bảng SinhVien
   SELECT HoTen FROM SinhVien WHERE MaSV = 'SV00000123';
   -- Xin cấp khóa Write Lock trên bảng Phong
   UPDATE Phong SET TinhTrang = N'Đang sử dụng' WHERE MaPhong = 'TN001-05';
   -- Toàn bộ khóa chỉ được giải phóng TỰ ĐỘNG tại đây
COMMIT TRANSACTION;
```

5.2 Giao thức Nhãn thời gian (Timestamp Ordering – TO)

Mỗi giao tác Ti được cấp một nhãn thời gian duy nhất TS(Ti) khi vào hệ thống. Mỗi mục dữ liệu X (ví dụ Phong.TinhTrang) duy trì 2 chỉ số: Read_TS(X) — nhãn thời gian lớn nhất của giao tác đã đọc X thành công, và Write_TS(X) — nhãn thời gian lớn nhất của giao tác đã ghi X thành công.

1. Giao tác Ti phát lệnh Read(X): nếu TS(Ti) < Write_TS(X) thì Ti muốn đọc giá trị đã bị đè bởi một giao tác tương lai ⇒ Abort & Rollback Ti; ngược lại thực hiện Read(X) và cập nhật Read_TS(X) = max(Read_TS(X), TS(Ti)).
2. Giao tác Ti phát lệnh Write(X): nếu TS(Ti) < Read_TS(X) hoặc TS(Ti) < Write_TS(X) thì giá trị Ti muốn ghi đã bị đọc/đè bởi giao tác tương lai ⇒ Abort & Rollback Ti; ngược lại thực hiện Write(X) và cập nhật Write_TS(X) = TS(Ti).

Áp dụng vào ví dụ Lost Update ở mục 4.2.1: nếu hệ thống dùng TO thay vì UPDLOCK, giao tác COMMIT trước (NV1, TS nhỏ hơn) sẽ khiến giao tác COMMIT sau (NV2) bị Abort ngay khi

Write(ChiSoCuoi) do TS(NV2) < Write_TS đã được NV1 cập nhật — thay vì âm thầm ghi đè như hiện trạng.

5.3 Luật Ghi Thomas (Thomas' Write Rule)

Trường hợp đặc biệt nâng cấp cho giao thức TO: khi Ti phát lệnh Write(X), nếu TS(Ti) < Write_TS(X), thay vì buộc Ti phải Abort/Rollback như TO thông thường, luật Thomas cho phép bỏ qua (ignore) thao tác Write(X) đó và cho Ti tiếp tục chạy — vì giá trị Ti định ghi đã bị ghi đè bởi một giao tác mới hơn nên việc bỏ qua không ảnh hưởng đến tính đúng đắn của dữ liệu cuối cùng.

5.4 Kỹ thuật Điều khiển Tương tranh Lạc quan (Optimistic Concurrency Control)

Phù hợp cho các màn hình tra cứu nhiều hơn cập nhật trong ứng dụng KTX (ví dụ màn hình Dashboard, tra cứu danh sách phòng/sinh viên). Gồm 3 giai đoạn:

1. Giai đoạn Đọc (Read Phase): đọc dữ liệu từ CSDL, các thao tác ghi chỉ thực hiện trên vùng nhớ tạm của giao tác.
2. Giai đoạn Kiểm tra (Validation Phase): kiểm tra xem trong quá trình Ti chạy có giao tác nào khác đã cập nhật trùng dữ liệu không (ví dụ so sánh giá trị ChiSoCuoi hiện tại với giá trị lúc đọc) — nếu phát hiện xung đột thì Abort.
3. Giai đoạn Ghi (Write Phase): nếu Validation thành công, ghi toàn bộ dữ liệu từ vùng nhớ tạm vào CSDL chính thức.

CHƯƠNG VI: CÁC KỸ THUẬT TỐI ƯU VÀ NÂNG CAO

6.1 Cơ chế Đa phiên bản (MVCC – Multi-Version Concurrency Control)

SQL Server hỗ trợ MVCC qua chế độ Snapshot Isolation / Read Committed Snapshot: hệ thống tạo ra các phiên bản (versions) dữ liệu khác nhau mỗi khi có thao tác ghi, thay vì khóa chặn thao tác đọc.

- MVCC dựa trên Timestamp: thao tác Đọc (Read) không bao giờ bị chặn bởi thao tác Ghi (Write) — sinh viên tra cứu hóa đơn hoặc tình trạng phòng trên ứng dụng vẫn mượt mà dù nhân viên đang cập nhật thanh toán hoặc lập hợp đồng cùng lúc.

```
Dữ liệu Phong.TinhTrang của TN001-05:
[Version 1] (TinhTrang = N'Trống',       TS = 10)
      --->  [Version 2] (TinhTrang = N'Đang sử dụng', TS = 20)
```

- MVCC dựa trên 2PL (Khóa Certify): bổ sung loại khóa thứ ba bên cạnh Read và Write Lock. Khi giao tác muốn Commit, phải chuyển từ Write Lock sang Certify Lock (không tương thích với bất kỳ khóa nào) để xác nhận chính thức phiên bản mới.

Khóa xin cấp \ Khóa đang giữ	**Read Lock**	**Write Lock**	**Certify Lock**
Read Lock	Có	Có	Không
Write Lock	Có	Không	Không
Certify Lock	Không	Không	Không

6.2 Giao thức Khóa dựa trên Cây (Tree-based Protocol)

Áp dụng lên cấu trúc phân cấp tự nhiên của KTX: ToaNha → Phong → HopDong. Quy tắc vận hành:

1. Một giao tác T có thể khóa bất kỳ nút nào đầu tiên (ví dụ khóa ToaNha TN001).
2. Các nút tiếp theo chỉ có thể bị khóa bởi T nếu nút cha của nó hiện đang bị khóa bởi T (T muốn khóa Phong TN001-05 thì phải đang giữ khóa trên ToaNha TN001).
3. Giải phóng khóa (Unlock) có thể thực hiện ở bất kỳ thời điểm nào.
4. Quy tắc không khôi phục khóa: nếu T đã Unlock một nút, T không bao giờ được phép đặt khóa lại trên chính nút đó nữa.

Ưu điểm khi áp dụng cho KTX: các giao tác thao tác trên các Tòa nhà khác nhau (ví dụ NV1 xử lý tòa TN001, NV2 xử lý tòa TN002) hoàn toàn độc lập, không tranh chấp khóa — khác với việc khóa toàn bộ bảng Phong như một số giải pháp đơn giản.

CHƯƠNG VII: ĐÁNH GIÁ TỔNG KẾT VÀ KẾT LUẬN

7.1 Bảng so sánh các giải pháp điều khiển tương tranh áp dụng cho QLKyTucXa

Giao thức	Ưu điểm	Nhược điểm	Trường hợp áp dụng trong KTX
Strict 2PL (mặc định SQL Server)	Tự động đảm bảo khả tuần tự xung đột, chống Hủy dây chuyền.	Dễ xảy ra Deadlock nếu các thủ tục khóa bảng theo thứ tự khác nhau (mục 4.2.5).	Các giao dịch tài chính chính xác tuyệt đối: lập hợp đồng, lập hóa đơn, ghi nhận thanh toán.
Timestamp Ordering (TO)	Không xảy ra Deadlock.	Tỷ lệ Abort & Rollback cao khi nhiều nhân viên thao tác cùng lúc.	Có thể cân nhắc cho các thao tác cập nhật đơn giản, tần suất tương tranh cao.
MVCC / Snapshot Isolation	Thao tác Đọc không bao giờ chặn thao tác Ghi.	Tốn tài nguyên lưu trữ các phiên bản cũ (tempdb).	Màn hình tra cứu, Dashboard cho sinh viên và ban quản lý — đọc nhiều, ít ghi.

7.2 Kết luận

1. Mô hình CSDL QLKyTucXa được chuẩn hóa với 12 bảng, ràng buộc toàn vẹn rõ ràng (PK/FK/CHECK) và các trigger nghiệp vụ, đáp ứng đầy đủ tính Độc lập Dữ liệu.
2. Bộ thủ tục T-SQL (sp_ThemHopDong, sp_TraPhong, sp_LapHoaDon, sp_ThanhToanHoaDon) xử lý các nghiệp vụ lập hợp đồng, trả phòng, lập hóa đơn và thanh toán trong khuôn khổ TRANSACTION, đảm bảo tính ACID.
3. Nhóm đã phân tích và minh chứng bằng kịch bản T-SQL cụ thể đầy đủ 5 vấn đề tương tranh theo đúng yêu cầu: Mất dữ liệu cập nhật, Đọc dữ liệu rác, Không đọc lại được dữ liệu, Bóng ma và Deadlock — mỗi vấn đề đều gắn với một tình huống nghiệp vụ thật của hệ thống Quản lý Ký túc xá, kèm giải pháp khắc phục cụ thể (UPDLOCK, mức cô lập REPEATABLE READ/SERIALIZABLE, quy ước thứ tự khóa).
4. Phương pháp chứng minh khả tuần tự bằng Precedence Graph và PolyGraph, cùng các giao thức khóa hiện đại (2PL, Timestamp Ordering, MVCC, Tree-based Protocol), được vận dụng nhất quán trên chính schema và nghiệp vụ của đề tài Quản lý Ký túc xá.